

ACADEMIA F5
Sistema de Gestión Académica

Memoria del proyecto
API REST (FastAPI + PostgreSQL) y frontend web (React + TypeScript)

Factoría F5 — Bootcamp de Inteligencia Artificial
Proyecto del Módulo 1: IA-Project-CRUD
Autoría: costanna
28/09/2026

Índice

1. Introducción	4
2. Objetivos	4
3. Tecnologías utilizadas	4
4. Arquitectura del sistema	5
4.1. Visión general	5
4.2. Arquitectura en capas (ADR 0001)	5
4.3. Estructura del repositorio	6
5. Modelo de datos	6
5.1. Decisiones de diseño	7
6. API REST	7
6.1. Resumen de endpoints	8
6.2. Códigos de error	9
7. Seguridad: autenticación y autorización	9
8. Funcionalidades avanzadas	10
8.1. Paginación, filtros y exportación a CSV	10
8.2. Caché en memoria	10
8.3. Notificaciones en tiempo real	10
8.4. Integración con un servicio externo: Resend	10
8.5. Logging y manejo de excepciones	11
9. Interfaz de usuario	11
10. Calidad y pruebas	11
11. Integración continua y despliegue	12
11.1. Pipeline de GitHub Actions	12
11.2. Docker	12
11.3. Despliegue en la nube	12
12. Gestión del proyecto	12
12.1. Metodología y roles	12
12.2. Sprints e historias de usuario	13

13. Retrospectiva	14
13.1. Qué funcionó bien	14
13.2. Qué no funcionó tan bien	14
13.3. Acciones de mejora	14
14. Resultados y conclusiones	14
14.1. Líneas futuras	15
Anexo. Puesta en marcha en local	15
Backend	15
Frontend	16
Tests	16

1. Introducción

Este documento es la memoria técnica del proyecto **Academia F5**. El planteamiento parte de un encargo realista: un equipo de desarrollo de una consultora tecnológica especializada en pymes recibe la petición de un centro educativo que necesita digitalizar y optimizar la gestión de estudiantes, profesores, cursos, matrículas y notas, sustituyendo las hojas de cálculo manuales con las que trabaja hoy.

La solución desarrollada consta de una **API REST** construida con FastAPI sobre una base de datos relacional PostgreSQL y de un **frontend web** (SPA en React con TypeScript) que permite el uso diario del sistema sin depender de Swagger. El proyecto se ha planificado con Scrum en dos sprints de una semana, más un sprint corto de cierre tras la primera demo.

2. Objetivos

El objetivo general es desarrollar una API REST y una base de datos SQL que permitan gestionar eficientemente la academia, preparando el sistema para crecer en número de cursos, usuarios y funcionalidades. De él se derivan los siguientes objetivos específicos:

- Diseñar un modelo relacional de al menos cinco tablas con relaciones 1:1, 1:N y N:M e integridad referencial.
- Ofrecer un CRUD completo de todos los recursos, con paginación, filtros, exportación a CSV y errores HTTP semánticos.
- Proteger la API con autenticación JWT y control de acceso por rol (administrador, profesor y estudiante) y por propiedad de los datos.
- Añadir funcionalidades avanzadas: caché, notificaciones en tiempo real con WebSocket e integración con un servicio externo de email.
- Garantizar la calidad con una suite de tests automatizada, linters y un pipeline de integración continua.
- Contenerizar la aplicación con Docker y desplegarla en la nube.

3. Tecnologías utilizadas

Capa	Tecnología
Backend	Python 3.12, FastAPI, SQLAlchemy 2.0, Alembic
Base de datos	PostgreSQL (producción) y SQLite (tests)

Capa	Tecnología
Autenticación	JWT (python-jose) y roles admin, teacher y student
Tiempo real	WebSockets (notificación de notas nuevas)
Servicios externos	Resend (email al estudiante cuando recibe una nota)
Frontend	React 18, TypeScript, Vite y React Router
Tests	pytest (unitarios e integración), Vitest + Testing Library, Playwright (end-to-end)
Calidad de código	black, isort, flake8 y pre-commit
CI/CD	GitHub Actions: lint y tests con PostgreSQL real, build del frontend y tests end-to-end
Contenerización	Docker y docker-compose (base de datos, backend y frontend)
Nube	Neon (PostgreSQL), Render (API) y Vercel (frontend)

4. Arquitectura del sistema

4.1. Visión general

El sistema sigue una arquitectura cliente-servidor. El navegador ejecuta la SPA de React, que consume la API REST mediante peticiones HTTP autenticadas con un token JWT y mantiene abierto un canal WebSocket para recibir notificaciones. La API accede a PostgreSQL a través de SQLAlchemy y, al registrar una nota, envía en segundo plano un email mediante Resend.

4.2. Arquitectura en capas (ADR 0001)

El backend se organiza en cuatro capas con una única dirección de dependencia, de arriba hacia abajo:

Capa	Responsabilidad
routers/	Capa HTTP: lectura de la petición, códigos de estado y autorización por endpoint.
services/	Lógica de negocio y orquestación: valida reglas y dispara notificaciones.
repositories/	Acceso a datos con el patrón Repository sobre SQLAlchemy.

Capa	Responsabilidad
models/ y schemas/	Entidades ORM (SQLAlchemy) y DTOs de entrada y salida (Pydantic).

Los routers nunca consultan la base de datos directamente y los servicios nunca construyen consultas a mano: usan un *BaseRepository* genérico con operaciones get, list, create, update y delete, ampliado con métodos específicos cuando es necesario. Las excepciones de dominio (NotFoundError, ConflictError, etc.) se lanzan en los servicios y se traducen a códigos HTTP en un único punto (app/exceptions.py y app/main.py).

Se descartó escribir toda la lógica en el router, porque obligaría a probarlo todo mediante tests de integración, y también el patrón Active Record, porque SQLAlchemy 2.0 favorece el patrón Data Mapper. El coste aceptado es algo más de código repetitivo a cambio de piezas pequeñas, previsibles y fáciles de testear de forma aislada.

4.3. Estructura del repositorio

```

backend/  API REST (FastAPI)
  app/core/      configuración, JWT, caché, logging
  app/models/    entidades SQLAlchemy (7 tablas)
  app/schemas/  DTOs Pydantic
  app/repositories/ acceso a datos
  app/services/  lógica de negocio
  app/routers/   endpoints REST y WebSocket
  alembic/      migraciones
  tests/        unit/ e integration/
frontend/  SPA React + Vite (e2e/ con Playwright)
docs/     diagrama ER, API, Kanban, retrospectiva, ADR
.github/workflows/ci.yml  pipeline de CI
docker-compose.yml       stack completo con un comando

```

5. Modelo de datos

La base de datos tiene siete tablas relacionadas. Existen relaciones 1:1 (User con Student y User con Teacher), 1:N (Teacher con Course, Course con Schedule y Enrollment con Grade) y una relación N:M entre Student y Course resuelta mediante la tabla Enrollment.

Tabla	Campos principales	Relaciones
users	id, email (único), hashed_password, role, is_active, created_at	1:1 con students y teachers
students	id, user_id, first_name, last_name, birth_date, phone, enrollment_date	1:N con enrollments
teachers	id, user_id, first_name, last_name, specialty, hire_date	1:N con courses
courses	id, name, description, credits, teacher_id (opcional)	1:N con schedules y enrollments
schedules	id, course_id, day_of_week, start_time, end_time, classroom	N:1 con courses
enrollments	id, student_id, course_id, enrollment_date, status	Tabla puente N:M; 1:N con grades
grades	id, enrollment_id, evaluation_name, score, date	N:1 con enrollments

5.1. Decisiones de diseño

- **Enrollment** no es solo una tabla técnica: guarda también el estado de la matrícula (activa, completada o abandonada).
- La restricción **UNIQUE(student_id, course_id)** impide matrículas duplicadas a nivel de base de datos, no solo en la aplicación.
- **teacher_id** en courses admite nulos con **ON DELETE SET NULL**: si se borra un profesor, sus cursos quedan sin profesor asignado.
- El resto de claves foráneas usan **ON DELETE CASCADE**, porque no tiene sentido conservar horarios, matrículas o notas huérfanas.
- Los modelos usan tipos SQL estándar, por lo que el mismo esquema funciona en PostgreSQL (producción) y en SQLite (tests).
- Las migraciones se versionan con Alembic y se han verificado contra PostgreSQL real, tanto en local como en Neon.

6. API REST

La API expone 33 endpoints REST bajo el prefijo /api/v1 y un canal WebSocket. FastAPI genera automáticamente la documentación interactiva en /docs (Swagger UI) y /redoc. Todos los endpoints, salvo /health, /auth/register y /auth/login,

requieren la cabecera Authorization: Bearer <token>.

6.1. Resumen de endpoints

Recurso	Método y ruta	Roles permitidos
Auth	POST /auth/register	Público (student); admin para crear admin o teacher, salvo la primera cuenta
Auth	POST /auth/login	Público; devuelve un JWT
Auth	GET /auth/me	Cualquier usuario autenticado
Estudiantes	GET /students, GET /students/{id}	admin, teacher
Estudiantes	GET /students/me	student (perfil propio)
Estudiantes	GET /students/export	admin, teacher (CSV)
Estudiantes	POST, PUT, DELETE /students	admin
Profesores	GET /teachers	Cualquier usuario autenticado
Profesores	GET /teachers/me	teacher (perfil propio)
Profesores	POST, PUT, DELETE /teachers	admin
Cursos	GET /courses	Cualquier usuario autenticado; filtro teacher_id; respuesta cacheada
Cursos	GET /courses/export	admin (CSV)
Cursos	POST, PUT, DELETE /courses	admin (invalida la caché)
Horarios	GET /schedules	Cualquier usuario autenticado; filtro course_id
Horarios	POST, DELETE /schedules	admin
Matrículas	GET /enrollments	Todos; cada estudiante ve las suyas y cada profesor, las de sus cursos
Matrículas	POST /enrollments	admin; teacher en sus cursos; student solo a sí mismo

Recurso	Método y ruta	Roles permitidos
Matrículas	PUT, DELETE /enrollments/{id}	admin; teacher en sus cursos
Notas	GET /grades	Todos; filtrado por propiedad como en matrículas
Notas	POST /grades	admin; teacher en sus cursos. Avisa por WebSocket y email
Notas	PUT, DELETE /grades/{id}	admin; teacher en sus cursos
WebSocket	WS /ws/notifications?token=. ..	Cualquier usuario autenticado
Salud	GET /health	Público

6.2. Códigos de error

Todas las respuestas de error siguen el formato {"detail": "mensaje"} y usan códigos HTTP semánticos:

Código	Significado
401	Token ausente o inválido, o credenciales incorrectas.
403	Usuario autenticado sin permiso: rol incorrecto, estudiante que actúa sobre datos ajenos o profesor sobre un curso que no imparte.
404	Recurso no encontrado.
409	Conflicto: email o matrícula duplicados.
422	Error de validación del cuerpo de la petición.
500	Error inesperado del servidor (queda registrado en el log).

7. Seguridad: autenticación y autorización

La autenticación se basa en **JWT firmados con HS256** (ADR 0002). El token se emite en POST /auth/login con el flujo estándar OAuth2PasswordBearer de FastAPI e incluye como claims el email del usuario y su rol. Al no depender de sesiones en el servidor, la API puede escalar horizontalmente. Las contraseñas se guardan con hash bcrypt.

La autorización por rol es declarativa: la dependencia `require_roles(*roles)` de `app/deps.py` se añade en la firma de cada ruta, de modo que se ve de un vistazo

quién puede llamarla. El mismo JWT se reutiliza en el WebSocket, pasándolo como parámetro de la URL, ya que los navegadores no permiten enviar la cabecera Authorization en ese canal.

En el tercer sprint se detectó que comprobar solo el rol dejaba huecos, así que se añadieron comprobaciones de **propiedad** de los datos:

- El registro público solo crea cuentas de estudiante; las cuentas de administrador y profesor las crea un administrador (salvo la primera).
- Un estudiante solo puede matricularse a sí mismo y solo ve sus propias matrículas y notas.
- Un profesor solo gestiona las matrículas y notas de los cursos que imparte.

Como contrapartida aceptada, un token no se puede revocar antes de que expire; por eso la expiración es corta (60 minutos por defecto, configurable). Las claves y credenciales se leen de variables de entorno con pydantic-settings y el archivo .env está excluido del control de versiones.

8. Funcionalidades avanzadas

8.1. Paginación, filtros y exportación a CSV

Los listados admiten los parámetros skip y limit y filtros por teacher_id, course_id, student_id y enrollment_id. Los estudiantes y los cursos pueden exportarse a CSV para informes externos, tanto desde la API como con un botón del frontend.

8.2. Caché en memoria

El listado de cursos se guarda en una caché en memoria (TTLCache) que se invalida explícitamente al crear, editar o borrar un curso. Resultó más simple y suficiente que montar Redis para el alcance del proyecto, y está aislada en app/core/cache.py para poder sustituirla si hace falta escalar a varias instancias.

8.3. Notificaciones en tiempo real

Cuando un profesor registra una nota, el estudiante conectado recibe al instante un mensaje new_grade por el WebSocket /ws/notifications. El frontend reconecta el canal automáticamente si se pierde la conexión.

8.4. Integración con un servicio externo: Resend

Al registrar una nota, el estudiante recibe también un email enviado con Resend. El envío se hace en segundo plano (BackgroundTasks) después de responder, y un fallo del proveedor solo se registra en el log, sin afectar a la petición. Si no se configura RESEND_API_KEY, el envío queda desactivado. El servicio

(EmailService) recibe el transporte por inyección, lo que permite testearlo sin red.

8.5. Logging y manejo de excepciones

La aplicación configura un logging básico centralizado y un manejador global de excepciones que traduce los errores de dominio a respuestas HTTP coherentes y registra los errores inesperados.

9. Interfaz de usuario

El frontend es una SPA en React 18 con TypeScript, construida con Vite y React Router. Sus principales características son:

- Inicio de sesión y rutas protegidas según el rol del usuario.
- Panel de inicio (dashboard) con las últimas notas y las notificaciones en tiempo real.
- Páginas de estudiantes, profesores, cursos, horarios, matrículas y notas con CRUD completo y edición en línea en todas las tablas.
- Relaciones mostradas por nombre (profesor de un curso, estudiante y curso de una matrícula) en lugar de identificadores.
- Paginación, filtros y exportación a CSV.

10. Calidad y pruebas

Nivel	Herramienta	Alcance
Backend	pytest	84 tests unitarios y de integración, 97 % de cobertura; incluye tests de permisos y del servicio de email con transporte simulado.
Frontend	Vitest y Testing Library	13 tests de componentes (login y CRUD).
End-to-end	Playwright	3 escenarios en navegador real: login, flujo profesor → curso → estudiante → matrícula → nota, y bloqueo por rol.

El código del backend se formatea con black e isort y se revisa con flake8. Los hooks de pre-commit ejecutan estas herramientas antes de cada commit, y el repositorio incluye .editorconfig y configuración de VS Code para mantener un estilo homogéneo.

Los tests encontraron errores reales durante el desarrollo, por ejemplo una incompatibilidad entre passlib y las versiones recientes de bcrypt, o un interceptor

de axios que recargaba la página de login e impedía ver el mensaje de credenciales incorrectas.

11. Integración continua y despliegue

11.1. Pipeline de GitHub Actions

Cada push y cada pull request ejecuta tres trabajos:

- **Backend:** flake8, comprobación de formato con black e isort, y tests contra un PostgreSQL real.
- **Frontend:** lint, tests unitarios y compilación con comprobación de tipos.
- **End-to-end:** levanta la API y el frontend y los recorre con Playwright en Chromium.

Desde la migración del tablero a GitHub Projects, cada cambio se desarrolla en su propia rama y se integra en main mediante una pull request con el CI en verde.

11.2. Docker

El archivo docker-compose.yml levanta todo el stack con un solo comando: PostgreSQL, el backend (que aplica las migraciones al arrancar) y el frontend servido como estático.

```
cp .env.example .env
docker compose up --build
```

11.3. Despliegue en la nube

Para la demo, la aplicación se desplegó con servicios gratuitos: **Neon** para PostgreSQL (con conexión agrupada), **Render** para la API a partir de la imagen Docker y un blueprint render.yaml, y **Vercel** para el frontend. El backend normaliza automáticamente la cadena de conexión de Neon y los orígenes CORS permitidos se configuran por variable de entorno. La guía completa está en docs/deployment.md.

12. Gestión del proyecto

12.1. Metodología y roles

El proyecto se ha desarrollado con una única persona como responsable técnica, pero organizado como si lo ejecutara un equipo pequeño, asumiendo explícitamente los roles de Scrum:

Rol	Responsabilidad
Product Owner	Prioriza el backlog, decide qué entra en cada sprint y acepta las historias frente a sus criterios.
Scrum Master	Vela por el timebox, elimina bloqueos y facilita la retrospectiva.
Desarrollo backend	Modelo de datos, API REST, autenticación y tests.
Desarrollo frontend	SPA en React, consumo de la API y experiencia de usuario.
Asistente de IA	Claude Code como pair programmer para acelerar el scaffolding, sugerir correcciones y generar documentación base. Toda sugerencia se revisa, se ejecuta y se valida: el criterio final es humano.

Las ceremonias fueron planning al inicio de cada sprint, seguimiento diario, review contra los criterios de aceptación y retrospectiva al cierre. La definición de hecho de cada historia es: endpoint implementado, tests en verde y documentación actualizada. Las decisiones de arquitectura quedan registradas como ADR en docs/adr/.

12.2. Sprints e historias de usuario

Sprint	Contenido	Historias
1. Fundación	Modelo de datos, autenticación, CRUD básico, tests y variables de entorno.	HU-01 a HU-06
2. Funcionalidad avanzada	Matrícula desde la web, notas con WebSocket, CSV, paginación, caché, CI, SPA y Docker.	HU-07 a HU-14
3. Cierre tras la demo	Edición en la web, asignación de profesores, permisos por propiedad, email con Resend, tests end-to-end y horarios.	HU-18 a HU-24
Backlog	Edición de horarios por el profesor, panel de estadísticas y recuperación de contraseña.	HU-15 a HU-17

Se completaron 21 de las 24 historias de usuario. El tablero Kanban se mantuvo en docs/kanban.md durante los sprints y después se migró a GitHub Projects, con un issue por historia.

13. Retrospectiva

13.1. Qué funcionó bien

- Definir el modelo de datos antes de escribir endpoints evitó retrabajo.
- La arquitectura en capas hizo que añadir endpoints fuera mecánico y fácil de testear.
- Escribir los tests junto con cada endpoint detectó errores reales de forma temprana.
- Desplegar pronto y probar la aplicación como usuario real destapó carencias que los tests no veían.

13.2. Qué no funcionó tan bien

- No fijar versiones de dependencias transitivas (bcrypt) costó una tarde de depuración.
- No prever el endpoint /students/me bloqueó temporalmente la matrícula de los propios estudiantes; además, el orden de declaración de rutas en FastAPI provocó errores 422 difíciles de diagnosticar.
- Revisar los permisos solo por rol dejó huecos de acceso a datos ajenos.
- El frontend se construyó inicialmente solo para el camino feliz (crear y listar) y se quedó corto frente al CRUD de la API.
- Configurar CORS a mano en Render requirió varias iteraciones.

13.3. Acciones de mejora

- Añadir a cada historia un criterio de aceptación de permisos: qué pasa si lo intenta otro rol u otro usuario.
- Decidir desde el diseño qué acciones necesita el propio usuario sobre sí mismo (/me).
- Mantener una lista de comprobación API frente a UI para no dejar endpoints sin pantalla.
- Registrar en el arranque la configuración crítica, como los orígenes CORS.

14. Resultados y conclusiones

Métrica	Valor
Tablas de base de datos	7, con migraciones versionadas en Alembic

Métrica	Valor
Endpoints	33 REST y 1 canal WebSocket
Tests	84 de backend (97 % de cobertura), 13 de frontend y 3 escenarios end-to-end
Servicios externos	Resend (email de notas)
Historias completadas	21 de 24

El proyecto cubre los cuatro niveles de entrega planteados: el esencial (siete tablas relacionadas, CRUD completo, tests, Kanban, variables de entorno, logging y excepciones), el medio (Swagger, errores semánticos, CSV, paginación y filtros), el avanzado (JWT con roles y control de propiedad, caché y WebSocket) y el experto (Docker, interfaz de usuario, despliegue real en la nube e integración con un servicio externo).

Como aprendizaje principal, el despliegue temprano y los tests end-to-end resultaron tan valiosos como los tests unitarios: fueron los que revelaron que la seguridad debía comprobar la propiedad de los datos y no solo el rol, y que la interfaz debía cubrir todo lo que ofrecía la API.

14.1. Líneas futuras

- Permitir a cada profesor editar los horarios de sus propios cursos.
- Panel de estadísticas con notas medias y ocupación de los cursos.
- Recuperación de contraseña por email, aprovechando la integración con Resend.
- Sustituir la caché en memoria por Redis si se escala a varias instancias, y añadir una lista de revocación de tokens.

Anexo. Puesta en marcha en local

Backend

```
cd backend
python -m venv .env
.venv\Scripts\activate
pip install -r requirements.txt
cp .env.example .env
alembic upgrade head
uvicorn app.main:app --reload
```

Frontend

```
cd frontend  
npm install  
cp .env.example .env  
npm run dev
```

Tests

```
cd backend && pip install -r requirements-dev.txt && pytest  
cd frontend && npm run test  
cd frontend && npx playwright install chromium && npm run test:e2e
```

Con el servidor en marcha, la documentación interactiva está en <http://localhost:8000/docs>. La primera cuenta del sistema puede registrarse como administrador; las siguientes cuentas de administrador o profesor las crea un administrador.